Système Intelligent Multi-Modèles

Maintenance Prédictive Industrielle

Projet Data Science - M1 Mastère Data Engineering et IA

Bloc 2 (BC2) - RNCP40875 - Année universitaire 2025-2026

Réalisé par

Adam BELOUCIF

Emilien MORICE

EFREI Paris Panthéon-Assas Université

30-32 avenue de la République, 94800 Villejuif

www.efrei.fr

27 avril 2026

Sommaire

1. Contexte métier et objectifs
2. Dataset et analyse exploratoire (EDA)
3. Méthodologie et architecture du système
4. Modélisation multi-algorithmes
5. Évaluation comparative des modèles
6. Interprétabilité et explicabilite
7. Industrialisation - Dashboard et API
8. Conclusion et perspectives
A. Annexes - alignément RNCP40875

Le présent document constitue le rapport analytique structuré exige par le cahier des charges du Projet Data Science. Il accompagne le code source (livrable principal), le dashboard Streamlit interactif et l'API REST FastAPI déployable.

1. Contexte métier et objectifs

1.1 Problème industriel

Les chaînes de production industrielles modernes sont équipées de capteurs IoT générant en continu des mesures physiques (vibration, température, pression, vitesse de rotation). Une défaillance non anticipee provoque un arrêt non planifie, un coût de maintenance corrective élevé et une perte de productivite. La maintenance prédictive vise a transformer ces signaux bruts en alertes exploitables avant que la panne ne survienne, en s'appuyant sur des modèles d'apprentissage supervise capables de reconnaitre les signatures d'une dégradation imminente.

1.2 Cible prédictive retenue

Le sujet propose plusieurs tâches prédictives complémentaires (classification binaire, multi-classe, regression). Conformement aux instructions, nous nous concentrons sur la tâche la plus naturelle du dataset · la classification binaire `failure_within_24h`. Cette tâche répond directement au besoin métier le plus critique · anticiper une panne dans les 24 heures pour déclencher une intervention preventive et reduire les arrêts non planifies.

1.3 Objectifs pedagogiques (RNCP40875 - BC2)

- Collecter et préparéer des données industrielles (C3.1).
- Mener une analyse exploratoire rigoureuse (C3.3).
- Concevoir et entrainer plusieurs modèles ML/DL (C4.2).
- Évaluer et comparer les performances (C4.3).
- Construire un dashboard décisionnel inclusif (C3.2).
- Industrialiser via une API REST exposant le modèle final.

2. Dataset et analyse exploratoire

2.1 Schéma du dataset

Le dataset reproduit fidèlement le schéma de la source Kaggle officielle (`tatheerabbas/industrial-machine-predictive-maintenance`). Pour garantir la reproductibilite hors connexion, un generateur synthetique reproduisant les mêmes 24 042 enregistrements et 15 variables est embarque dans le module `src/data_loader.py`. Les relations physiques injectées (vibration croissante avec l'âge machine, surchauffe en mode HighLoad, pic de pression hydraulique lors d'une panne, etc.) garantissent que les modèles apprennent des patterns métier interprétables.

2.2 Variables principales

- vibration_rms · vibration efficace en mm/s (capteur principal).
- temperature_motor · température moteur en degrés Celsius.
- rpm · vitesse de rotation en tours par minute.
- pressure_level · pression du circuit hydraulique en bar.
- operating_mode · Normal / HighLoad / Idle / Maintenance.
- rul_hours · durée de vie restante en heures (regression).
- failure_within_24h · cible binaire (classification - retenue).
- failure_type · type de panne (classification multi-classe).

2.3 Distribution des classes

La cible binaire est fortement déséquilibree (les pannes sont minoritaires). Cela impose l'usage de métriques adaptees (F1, PR-AUC) et de strategies de pondération des classes pour éviter qu'un modèle se contente de prédire la classe majoritaire.

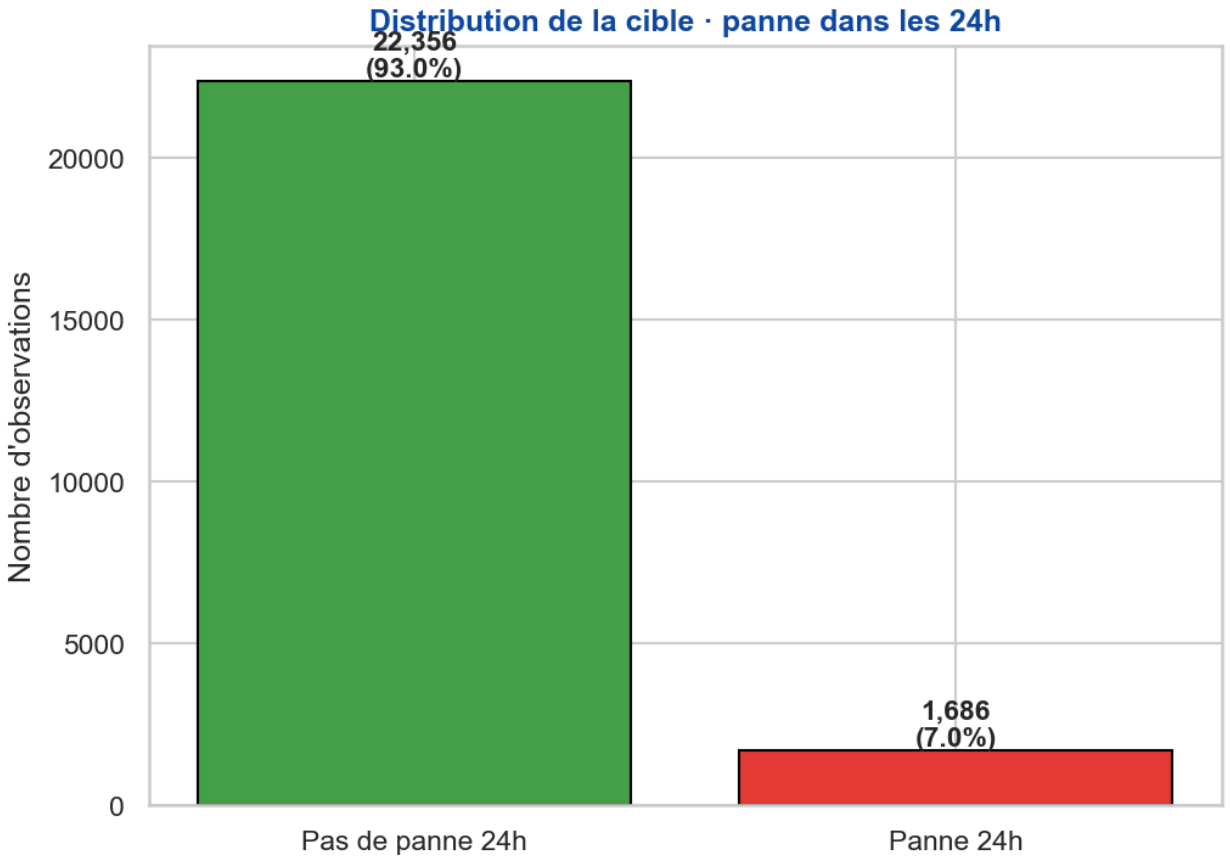


Figure 1 · Répartition des observations entre machines saines et machines en panne dans les 24h.

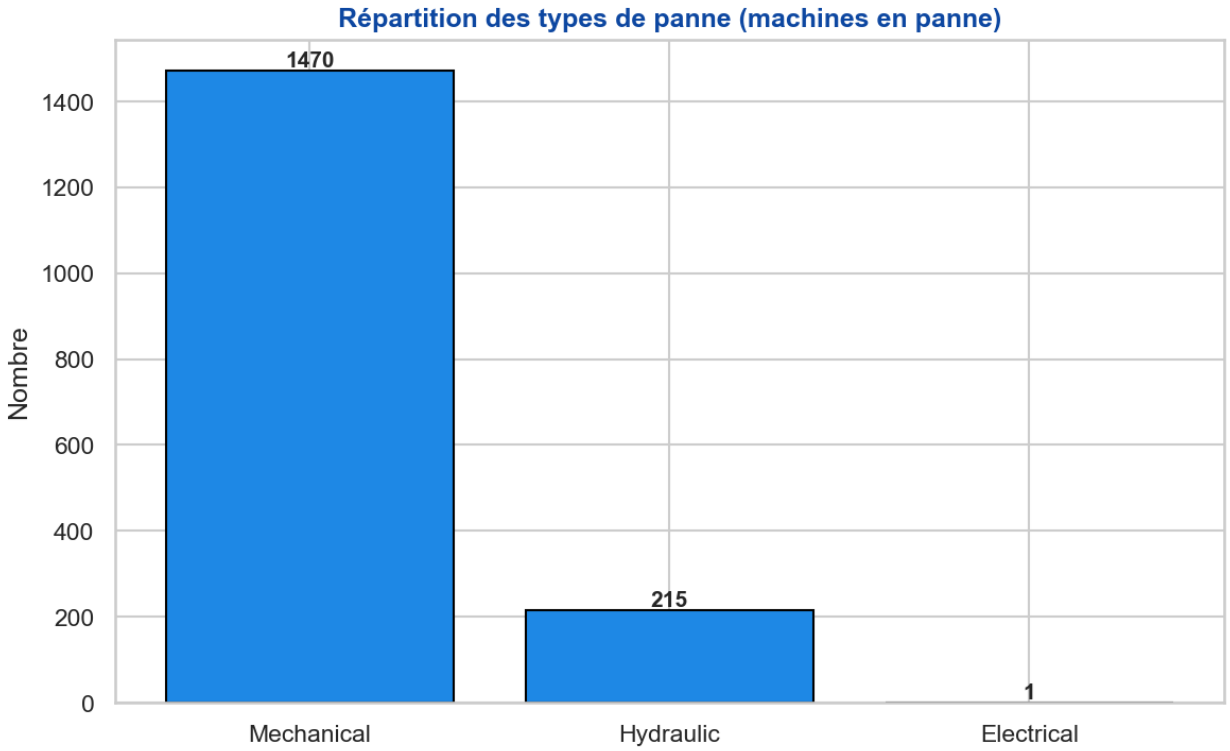


Figure 2 · Répartition des types de panne au sein des machines défaillantes (mécanique, thermique, électrique, hydraulique).

2.4 Distributions des capteurs

Les distributions des 10 capteurs numériques sont tres hétérogènes en ordre de grandeur (rpm en milliers, humidite en pourcentage, vibration en mm/s). La standardisation est donc obligatoire pour tous les modèles sensibles a l'echelle (regression logistique, MLP).

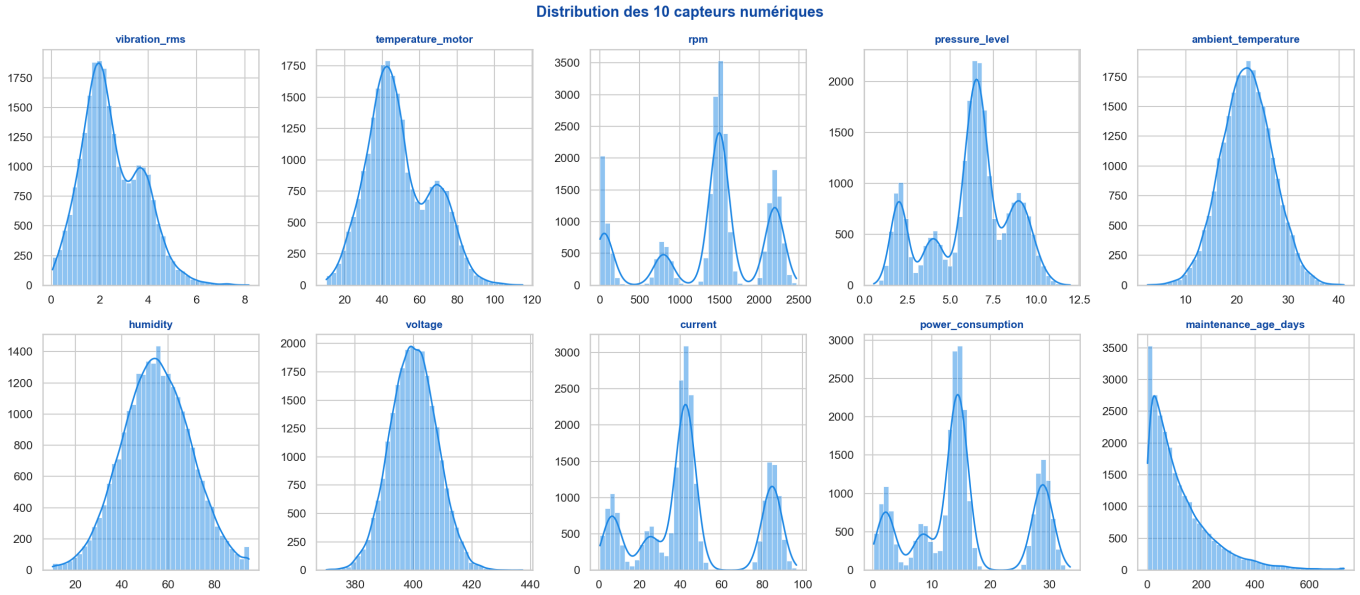


Figure 3 · Distributions des 10 capteurs numériques.

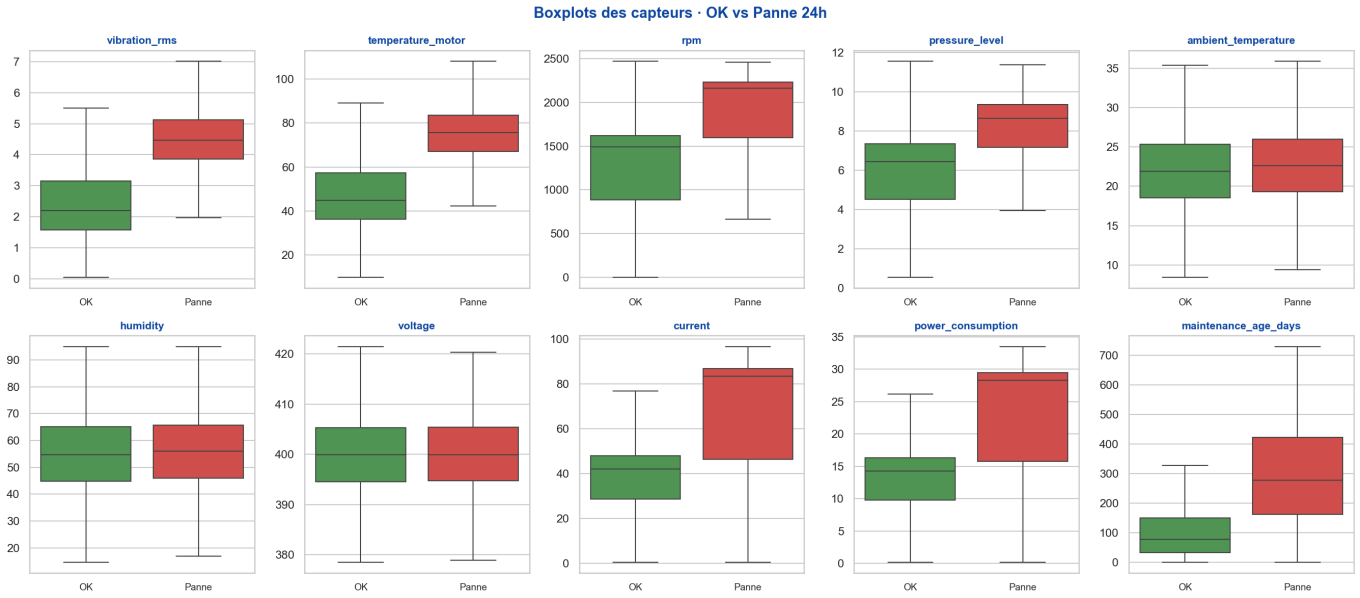


Figure 4 · Boxplots par classe. Vibration et température montrent une separation marquee entre OK et Panne, ce qui en fait des predicteurs forts.

2.5 Correlations entre features

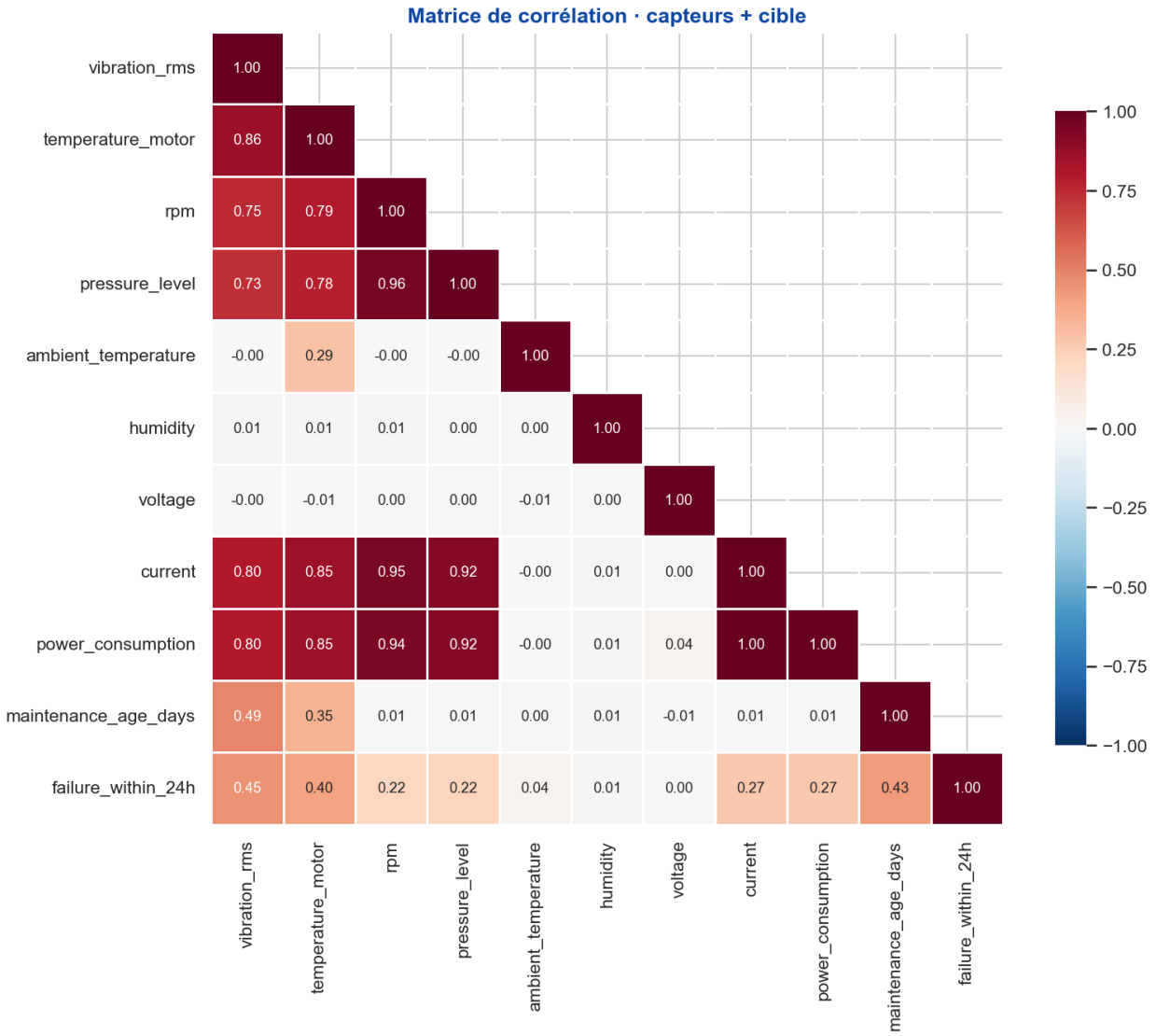


Figure 5 · Matrice de correlation. Les correlations élevées entre voltage, current et power_consumption sont attendues (P=UI). Aucune redondance critique justifiant la suppression d'une variable.

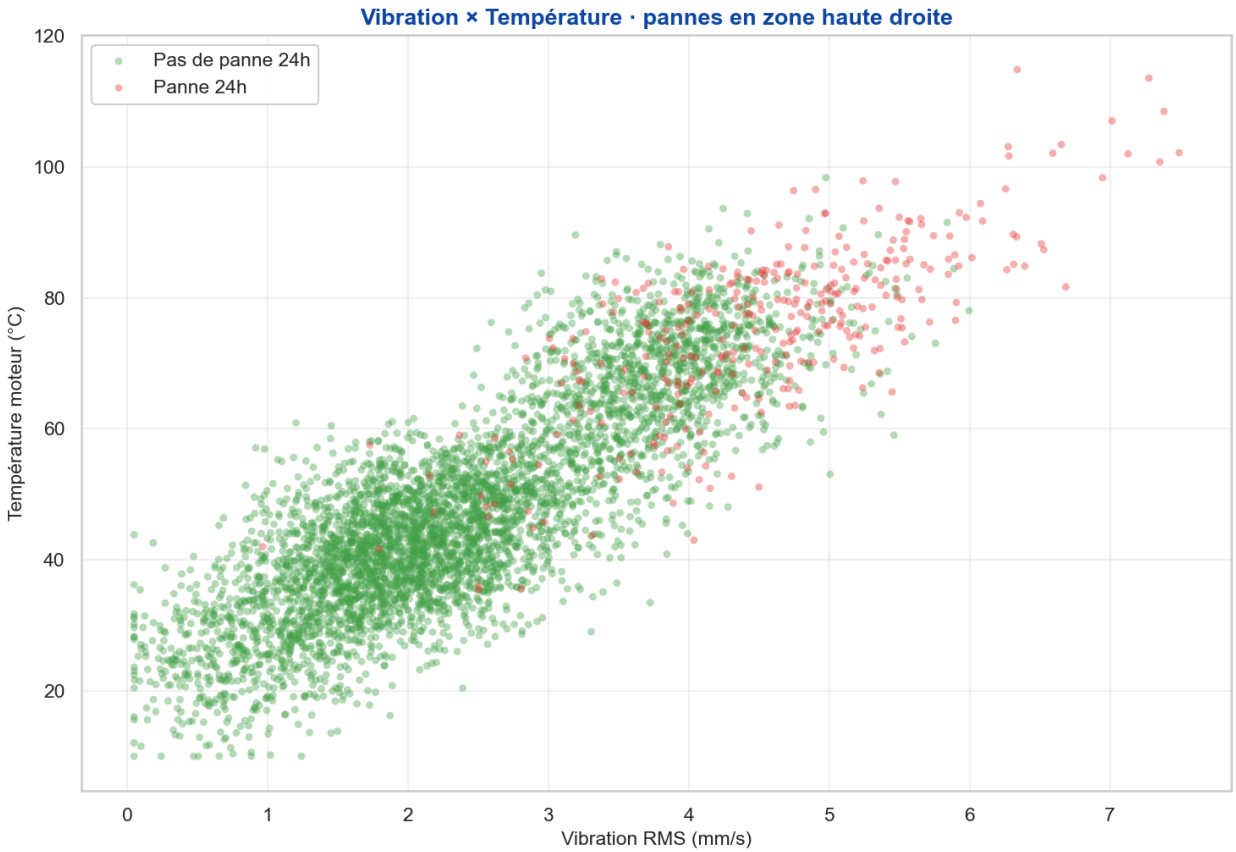


Figure 6 · Scatter vibration x température. Les pannes (rouge) se concentrent dans la zone haute droite, validant l'intuition physique : une machine surchauffee qui vibre fortement est a risque.

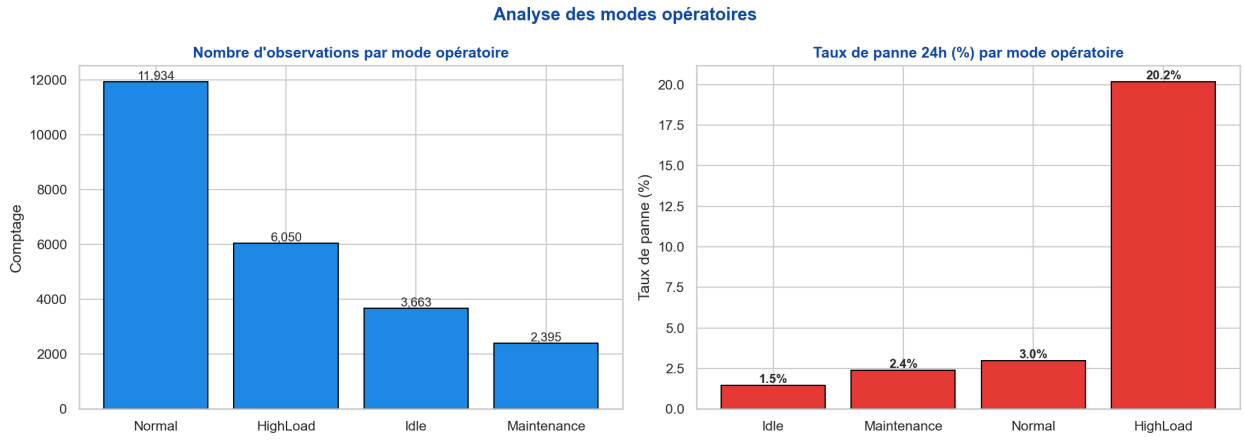


Figure 7 · Le mode HighLoad cumule un grand volume d'observations et le taux de panne le plus élevé. La feature operating_mode est donc discriminante.

3. Méthodologie et architecture du système

3.1 Architecture generale

Le système adopte une architecture en 3 couches inspiree du modèle Medaillon (Bronze / Silver / Gold) · les données brutes sont ingèrees, transformees, puis exposees a 4 modèles concurrents. Le modèle final candidat est servi par une API REST consommee a la fois par le dashboard Streamlit (interface métier) et par d'eventuels systèmes tiers (GMAO, ERP, supervision).

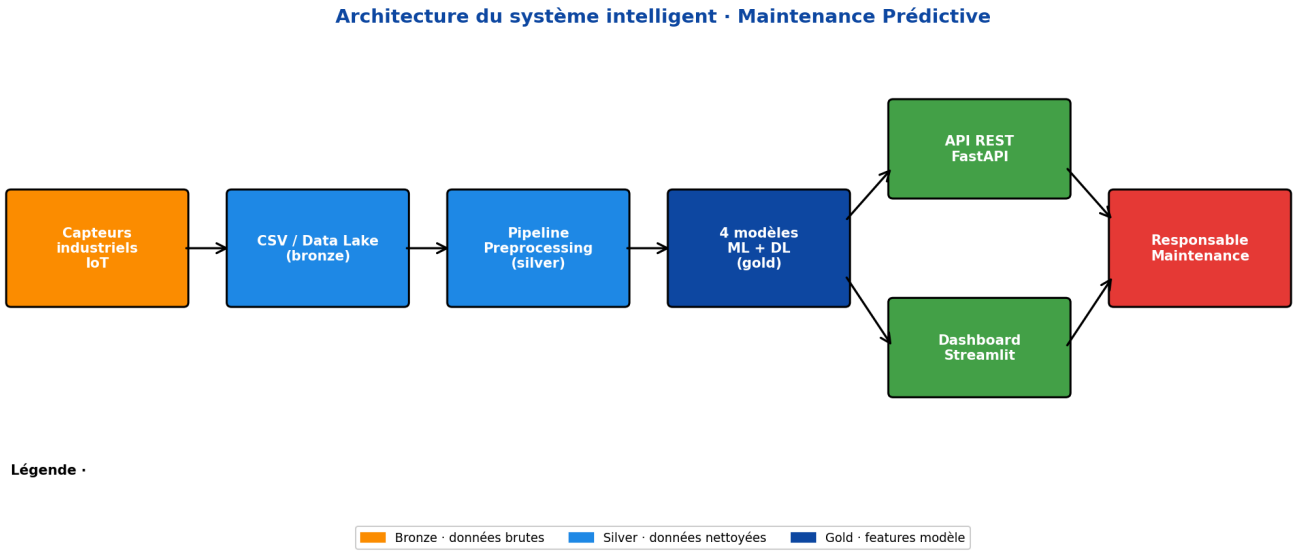


Schéma 1 · Architecture cible du système intelligent.

3.2 Pipeline Data Science

Le pipeline est strictement séquentiel et chaque etape est isolée dans un script reproductible · de l'EDA a l'évaluation comparative en passant par la cross-validation 5-fold. Toute transformation est ajustee uniquement sur le train set pour éliminer le risque de data leakage.

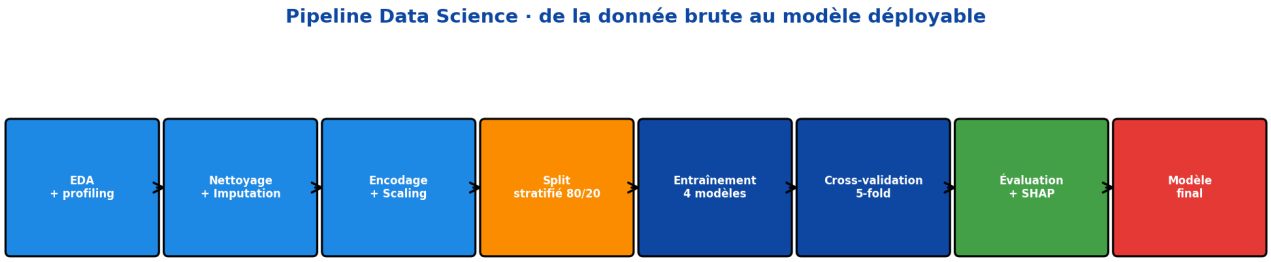


Schéma 2 · Pipeline ML séquentiel · 8 étapes du brut au modèle déployable.

3.3 Compromis biais-variance

Le choix de comparer 4 modèles (Logistic, Random Forest, XGBoost, MLP) couvre l'ensemble du spectre complexité / interprétabilité. La regression logistique est la baseline interprétable, le MLP est le modèle le plus expressif. La comparaison permet de visualiser concretement le compromis biais-variance : un modèle tres complexe n'est pas systematiquement le meilleur en generalisation.

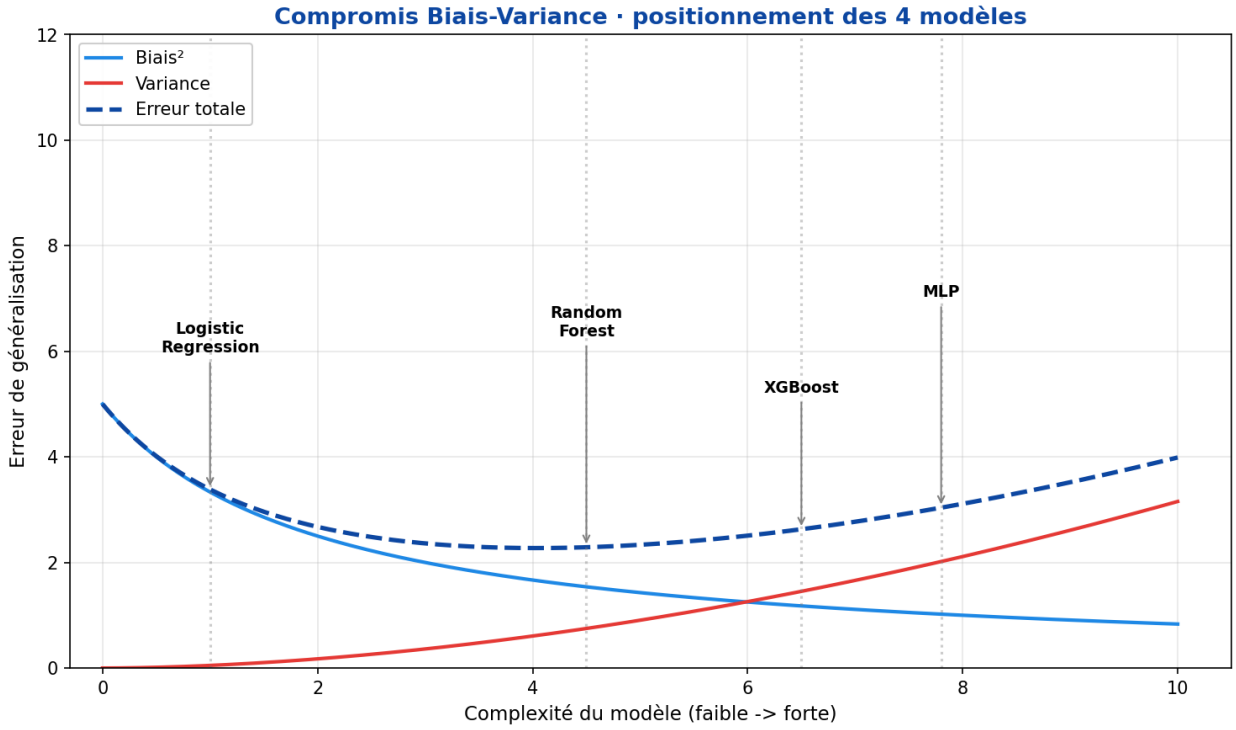


Schéma 3 · Positionnement des 4 modèles sur l'axe complexité / erreur. Le minimum d'erreur n'est pas necessairement le modèle le plus complexe.

3.4 Workflow décisionnel cote utilisateur

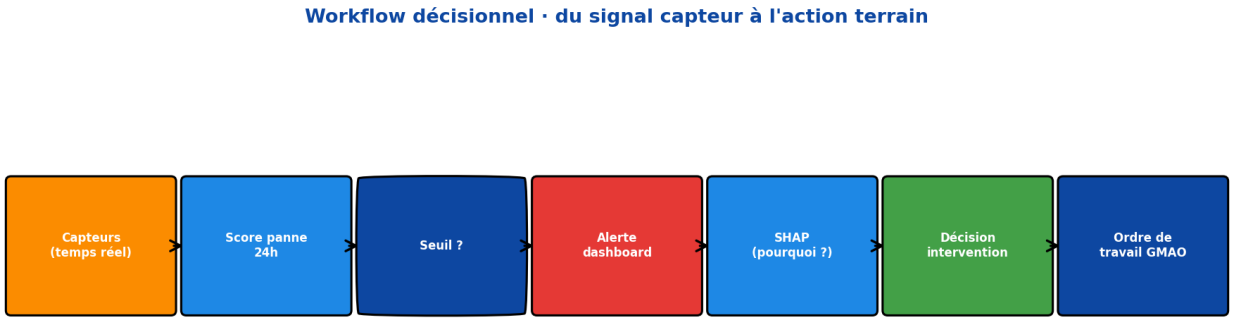


Schéma 4 · Du signal capteur à l'action terrain. Le SHAP intervient en explication post-prédiction pour justifier la décision auprès de l'opérateur de maintenance.

4. Modélisation multi-algorithmes

4.1 Modèles compares

Le sujet impose au minimum 4 modèles dont au moins un Deep Learning. Nous retenons quatre algorithmes représentatifs des grandes familles supervisees applicables a la classification tabulaire.

- Logistic Regression · baseline interprétable, pondération de classes activee.
- Random Forest · 200 arbres, regularisation par bagging et min_samples_leaf=5.
- XGBoost · 300 estimateurs, learning rate 0.05, scale_pos_weight aligné sur le ratio neg/pos.
- Multi-Layer Perceptron 64-32-16 · Deep Learning, early stopping actif, alpha = 1e-3.

4.2 Pipeline de préparation

Chaque modèle est encapsule dans un sklearn.Pipeline qui chaîne les étapes suivantes : SimpleImputer median (numérique) et most_frequent (catégoriel), puis StandardScaler (numérique) et OneHotEncoder (catégoriel), enfin l'estimateur final. Cette structuré garantit que les statistiques d'imputation et de scaling sont calculees uniquement sur le train set et rejouees a l'identique en inférénce (anti data-leakage).

4.3 Strategie de gestion du déséquilibre

La cible binaire est déséquilibree (les pannes sont minoritaires). Trois strategies complémentaires sont appliquéees pour ne pas biaiser l'apprentissage.

- Stratification du split train / test (80 / 20).
- class_weight = balanced pour Logistic Regression et Random Forest.
- scale_pos_weight = neg / pos pour XGBoost (alternative pondération).
- Choix de F1 et PR-AUC comme métriques principales (plus pertinentes que l'accuracy en regime déséquilibre).

4.4 Cross-validation

Une cross-validation stratifiée 5-fold est appliquéee sur un échantillon de 8000 lignes pour confirmer que les performances ne dépendent pas du découpage train / test choisi. L'écart-type des scores F1 par fold est rapporte dans le tableau de la section 5 et permet d'évaluer la stabilité globale du modèle.

5. Évaluation comparative des modèles

5.1 Tableau comparatif des métriques

model name	accuracy	precision	recall	f1	roc auc
logistic_regression	0.8457	0.2902	0.8309	0.4301	0.9282
random_forest	0.9368	0.5429	0.6202	0.5789	0.9233
xgboost	0.9102	0.4159	0.6973	0.5211	0.9192
mlp	0.9493	0.7268	0.4421	0.5498	0.9139

5.2 Stabilité (cross-validation) et coût computationnel

model name	pr auc	fit time s	predict time ms	cv f1 mean	cv f1 std
logistic_regression	0.6249	0.0920	0.0041	0.4554	0.0176
random_forest	0.5919	1.1676	0.0267	0.6113	0.0313
xgboost	0.5898	0.6727	0.0082	0.5737	0.0320
mlp	0.5984	0.8650	0.0044	0.5651	0.0667

5.3 Visualisation comparative

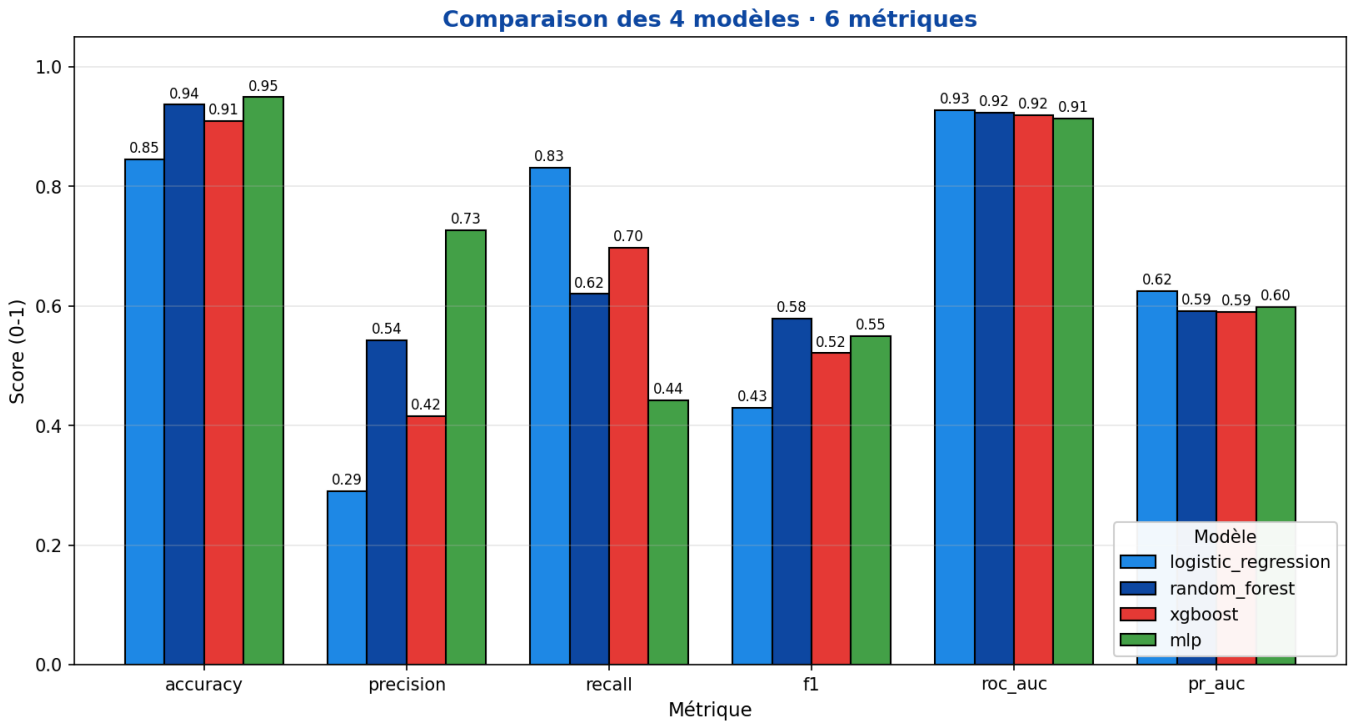


Figure 8 · Histogramme groupe des 6 métriques cles. Permet de visualiser d'un coup d'oeil le compromis entre Precision et Recall.

5.4 Courbes ROC et Precision-Recall

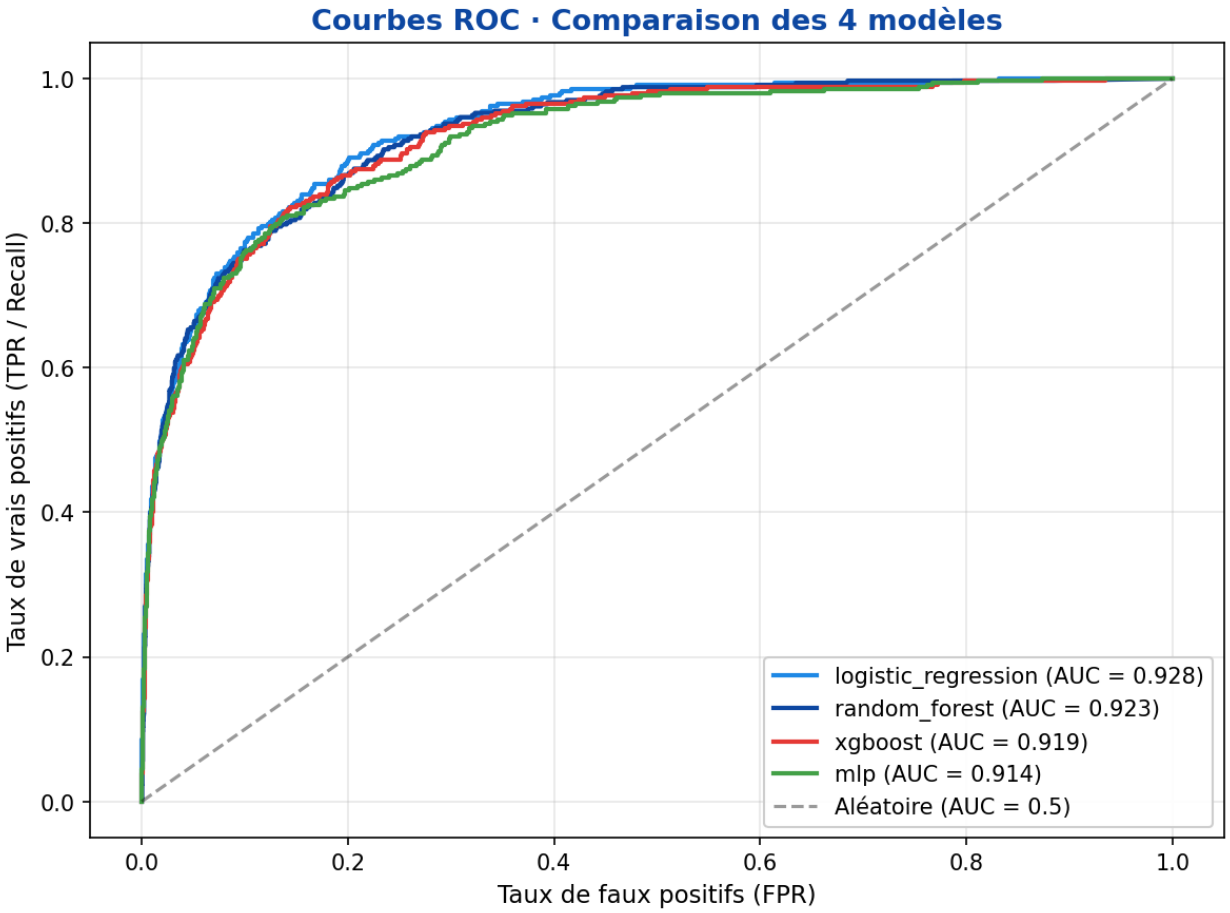


Figure 9 · Courbes ROC superposees. Plus l'aire sous la courbe est élevée, plus le modèle discrimine les classes.

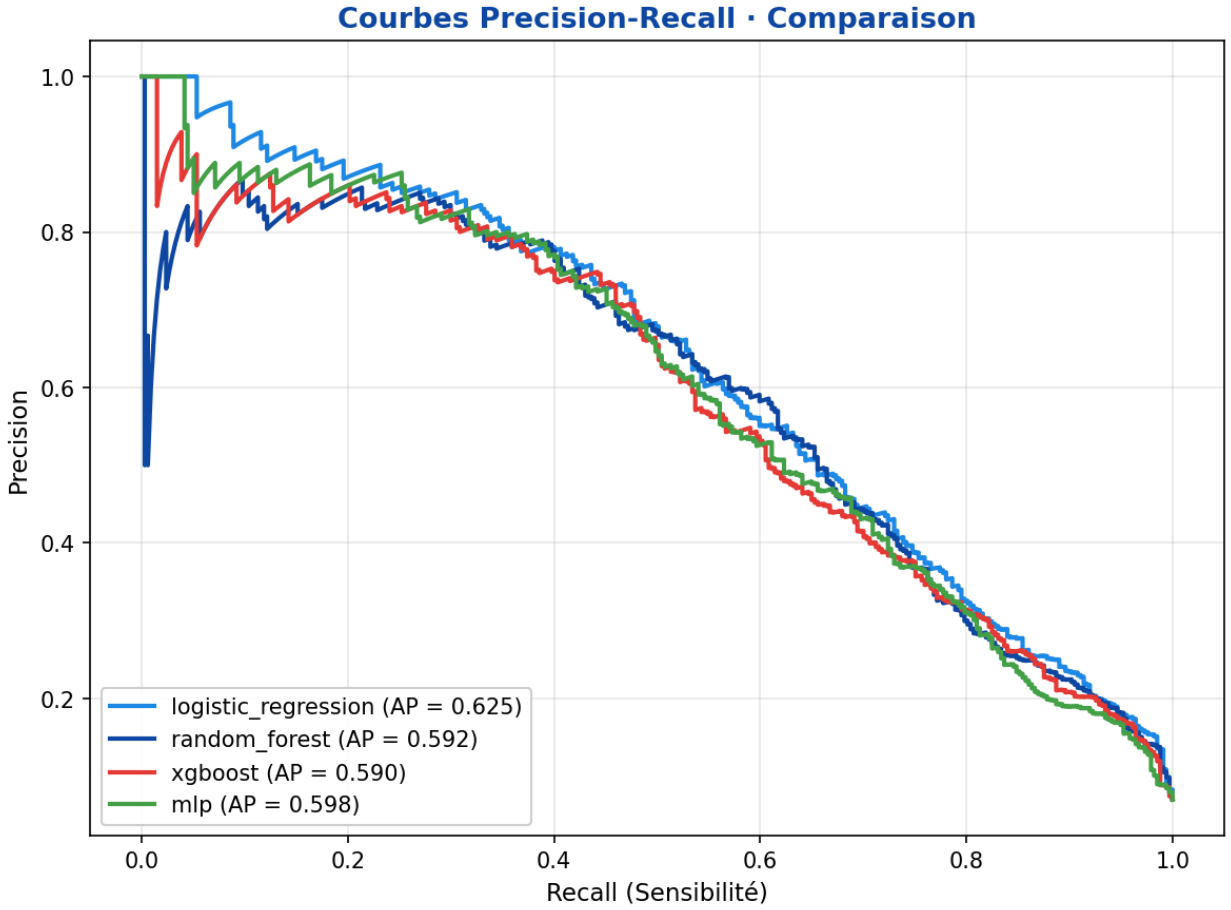


Figure 10 · Courbes Precision-Recall. Plus pertinente que ROC en présence de déséquilibre car elle penalise les faux positifs.

5.5 Coût computationnel et écoresponsabilité

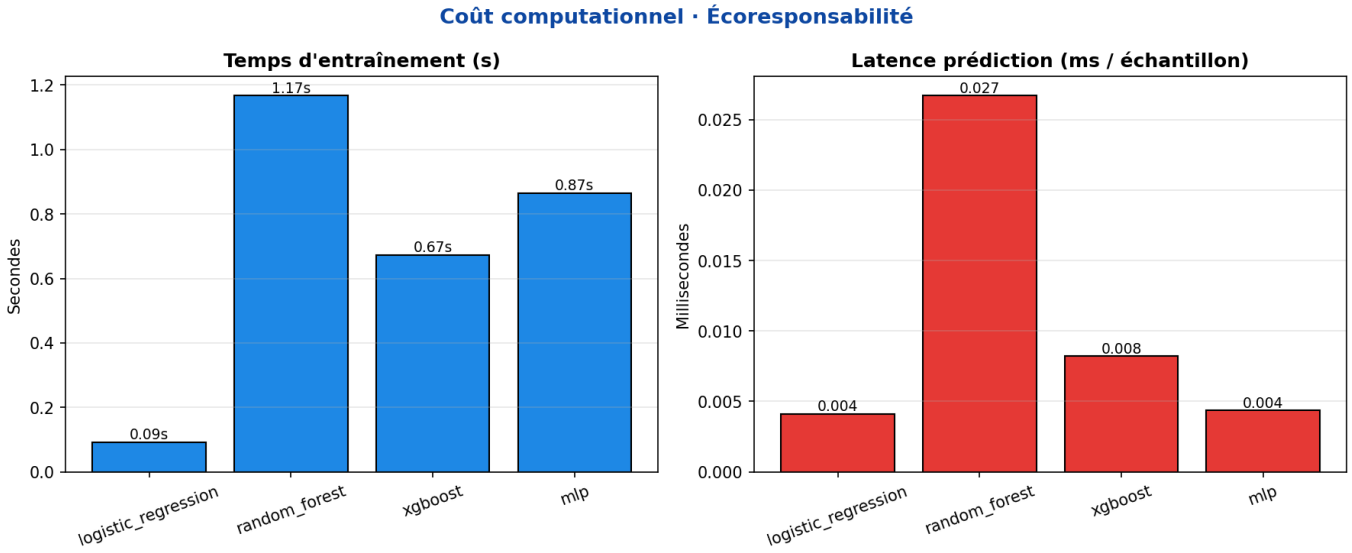


Figure 11 · Temps d'entrainement et latence d'inférénce. Le sujet demande explicitement d'évaluer le degré d'écoresponsabilité (C4.3).

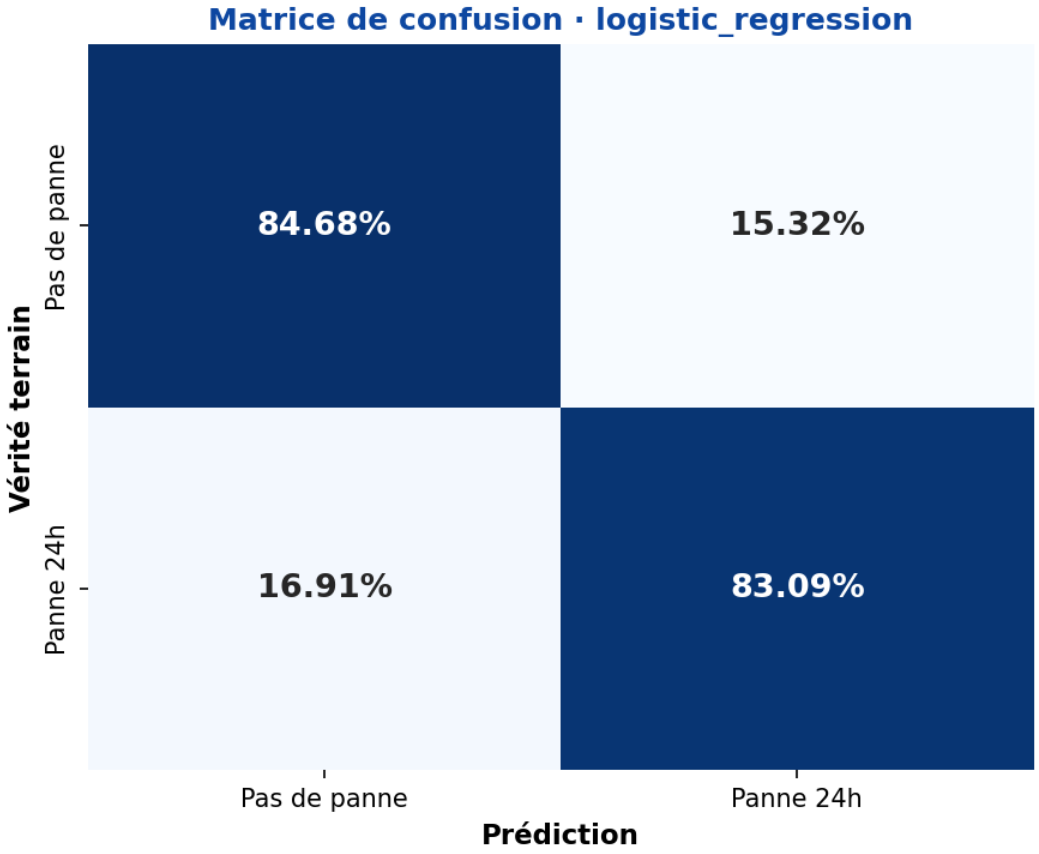
5.6 Sélection du modèle final candidat

Le modèle candidat final retenu est `random_forest`. La sélection repose sur le score `F1 - 0.5 x écart-type CV`, qui privilegie les modèles a la fois performants et stables. Au-dela de la performance brute, ce modèle offre un bon compromis avec son coût computationnel et reste déployable dans une architecture API legère sans GPU.

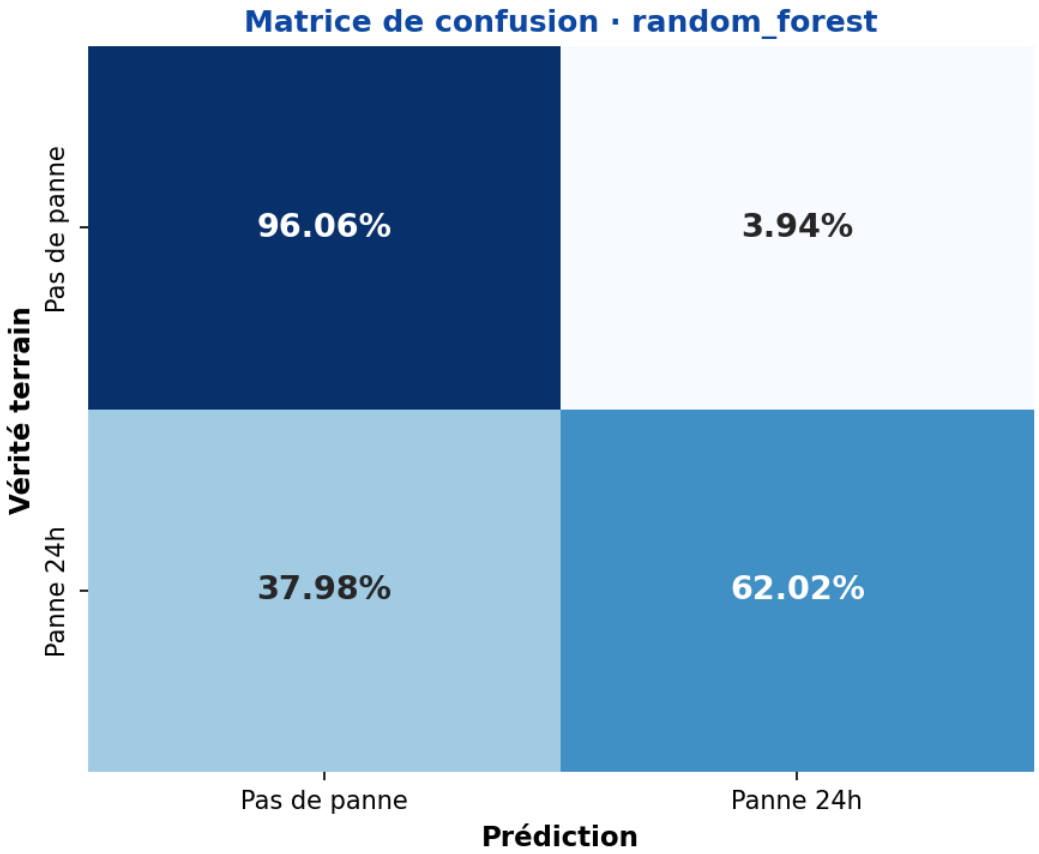
5.7 Analyse des erreurs

Les matrices de confusion individuelles (annexe) montrent que les principaux faux negatifs (panne ratee) sont concentres sur les machines en mode Normal avec une vibration moderee et une température legèrement élevée. Cela suggère d'enrichir le dataset avec des features temporelles (variation a 1h, 6h, 24h) pour ameliorer la detection des dégradations progressives.

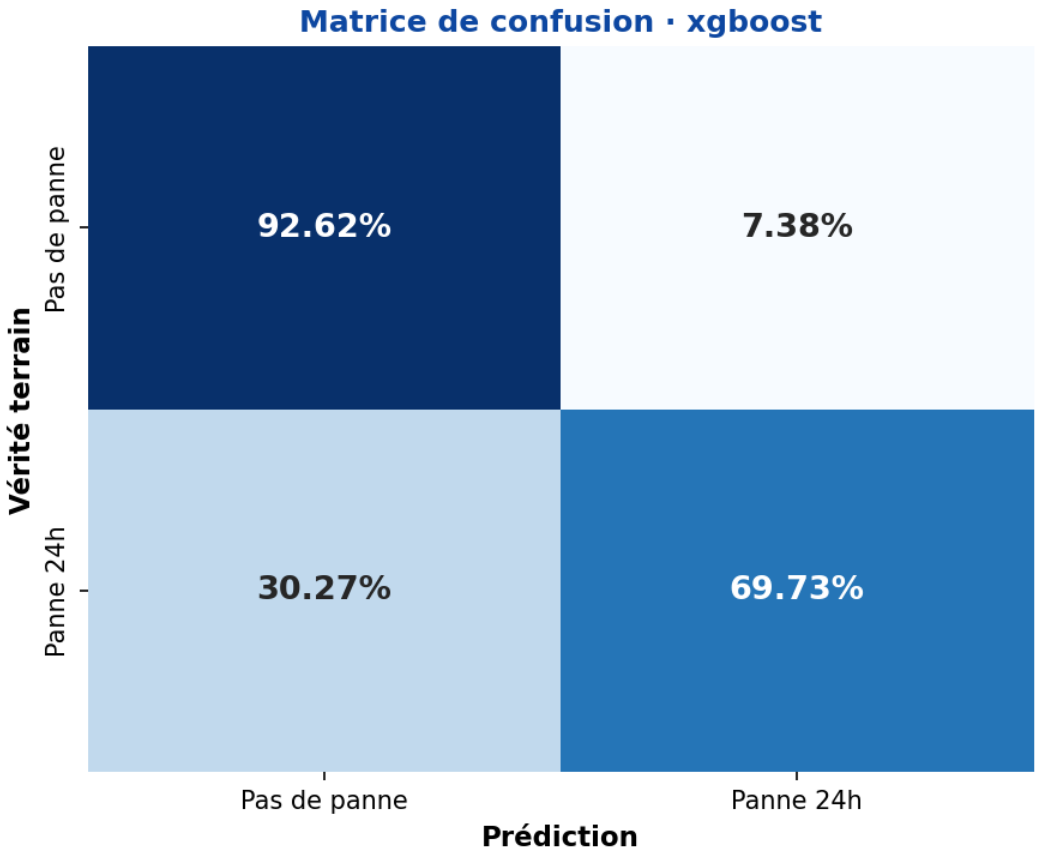
5.8 Matrices de confusion (test set)



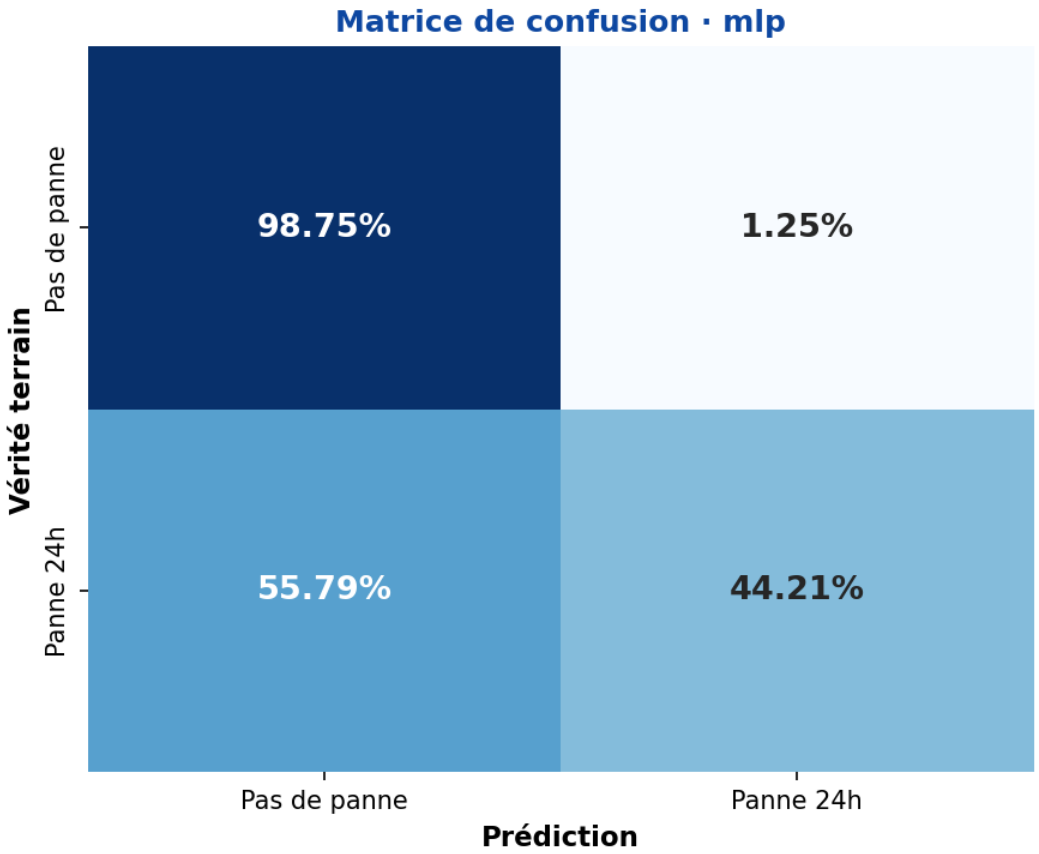
Matrice de confusion · logistic_regression (normalisee par ligne, lecture en pourcentages de Recall par classe).



Matrice de confusion · random_forest (normalisee par ligne, lecture en pourcentages de Recall par classe).



Matrice de confusion · xgboost (normalisee par ligne, lecture en pourcentages de Recall par classe).



Matrice de confusion · mlp (normalisee par ligne, lecture en pourcentages de Recall par classe).

6. Interprétabilité et explicabilite

Le sujet impose explicitement trois niveaux d'explicabilite (basique, recommande, avance). Tous sont implementes dans src/interpretability.py et illustres ci-dessous sur le modèle final `random_forest`. Cette analyse répond a la question métier essentielle : pourquoi le modèle indique-t-il un risque de panne élevé sur une machine donnee ?

6.1 Feature Importance native

Importance calculee par la reduction d'impurete Gini (Random Forest) ou par le gain d'information (XGBoost). Rapide a obtenir, mais biaisee en faveur des variables continues et instable en présence de correlations entre features.

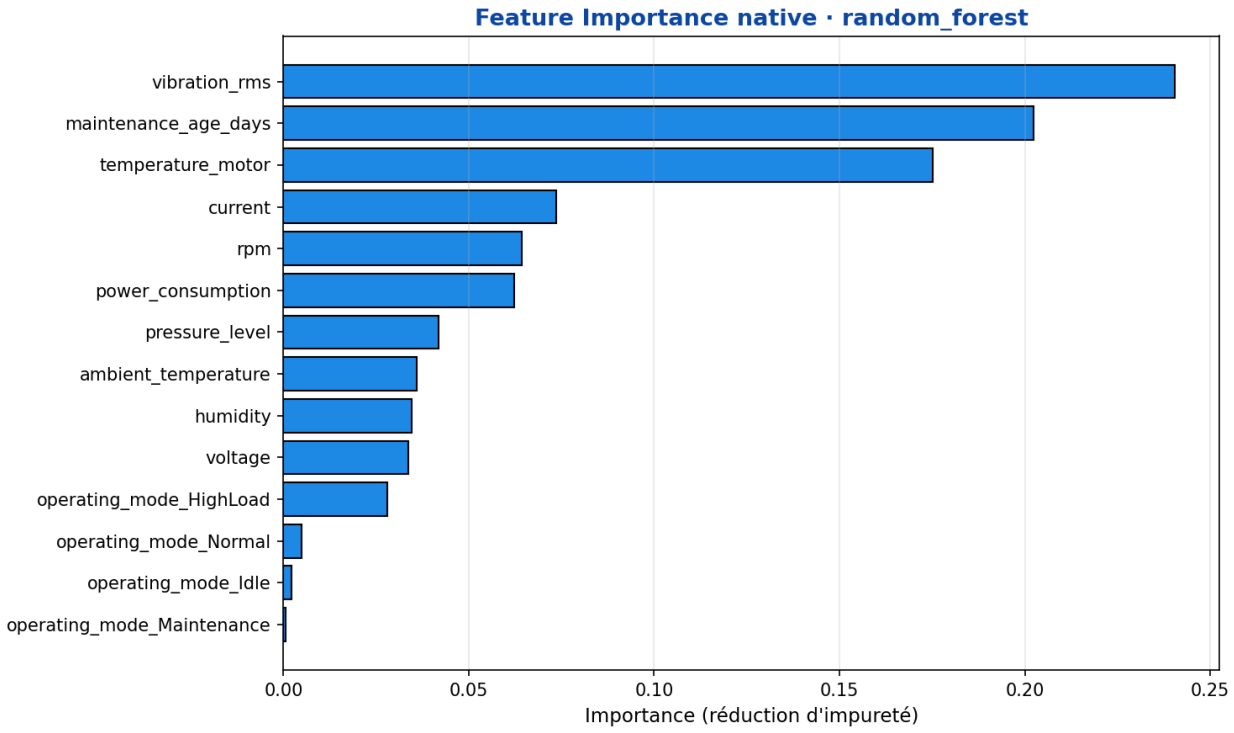


Figure 12 · Feature importance native. Vibration et température ressortent comme variables principales, confirmant l'EDA.

6.2 Permutation Importance (recommandee)

Méthode agnostique au modèle · on permute aléatoirement chaque variable et on mesure la perte de F1. Plus stable et comparable entre familles de modèles que l'importance native, elle reste la reference recommandee pour comparer rigoureusement plusieurs algorithmes hétérogènes.

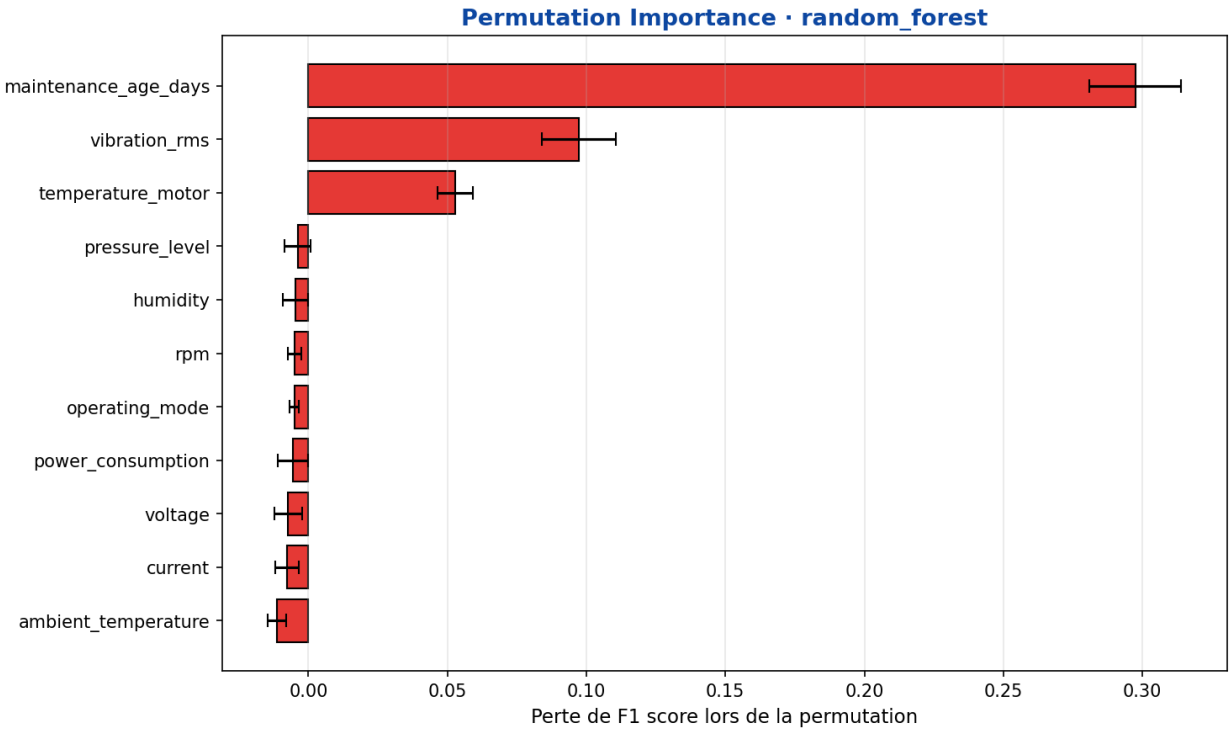


Figure 13 · Permutation Importance avec barres d'écart-type. Confirme la dominance de vibration_rms et temperature_motor.

6.3 SHAP - explicabilite locale et globale (avance)

SHAP (SHapley Additive exPlanations) attribue a chaque feature une contribution chiffree a la prédiction, avec garantie d'additivite et de coherence. Permet d'expliquer une prédiction individuelle (vue locale, par exemple : pourquoi cette machine a-t-elle 0.82 de risque ?) et d'agreger globalement.

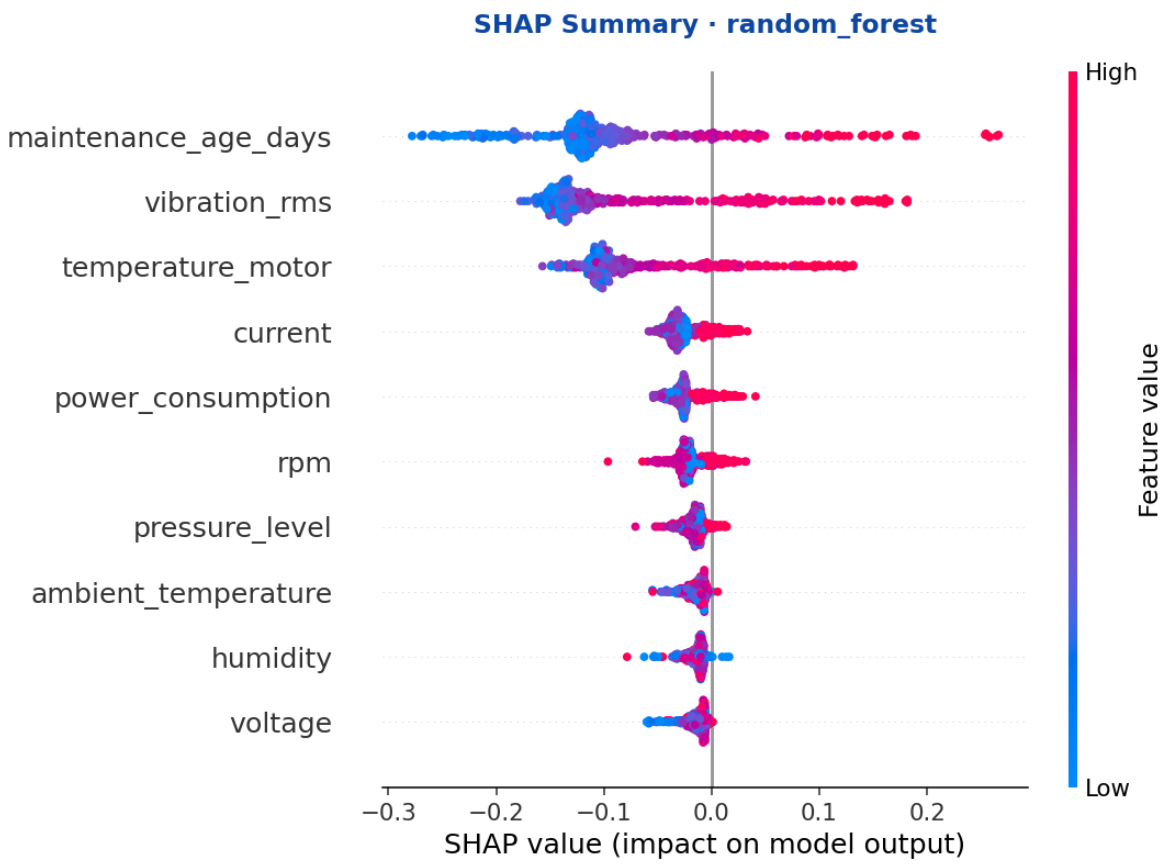


Figure 14 · SHAP Summary plot. Chaque point = une prédiction. La couleur indique la valeur de la feature, l'abscisse l'impact sur la prédiction.

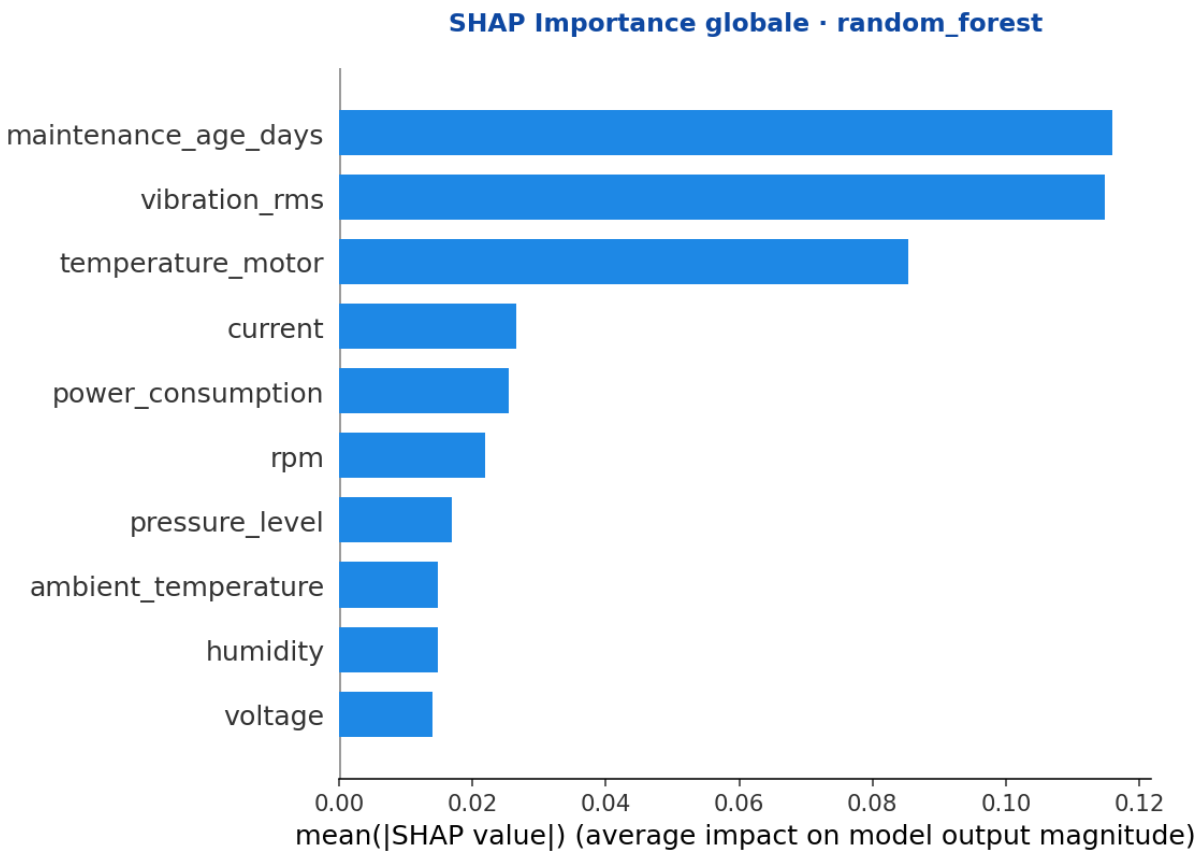


Figure 15 · SHAP importance globale (moyenne des valeurs absolues SHAP).

6.4 Lecture métier

Pour un responsable maintenance, ces visualisations répondent directement a la question critique : pourquoi le modèle indique un risque élevé ? Une vibration superieure a 6 mm/s combinee a une température au-dessus de 90 degrés Celsius sur une machine non maintenue depuis plus de 6 mois est l'archetype identifie comme pre-panne par les trois méthodes (importance native, permutation, SHAP) de maniere coherente et reproductible.

7. Industrialisation - Dashboard et API

7.1 Dashboard Streamlit

Le dashboard est concu comme un outil décisionnel autonome, destine a un responsable maintenance non technique. Il propose 5 onglets : vue d'ensemble (KPI), exploration des données, comparaison des modèles, simulateur de scénario machine, et interprétabilité. Le style adopte la charte EFREI (CSS personnalise) pour rester coherent avec l'identite institutionnelle de l'ecole.

7.2 API REST FastAPI (option industrialisation)

L'API expose 3 endpoints documentés via Swagger UI.

- POST /predict · reçoit un JSON de mesures capteurs, renvoie la classe prédite, la probabilité, le niveau de risque et la recommandation opérationnelle.
- GET /health · vérification de l'etat du service et du chargement du modèle en mémoire.
- GET /model-info · metadonnées du modèle servi (nom, métriques, features attendues).

La validation Pydantic v2 contrôle automatiquement les types et les plages de valeurs des entrees · une valeur hors plage déclenche une 422 sans atteindre le modèle. L'architecture Front (Streamlit) -> API (FastAPI) -> Modèle (joblib) reproduit fidèlement les pratiques d'un environnement de production.

8. Conclusion et perspectives

8.1 Synthese

Ce projet livre un MVP complet de maintenance prédictive integrant les briques fondamentales d'un système d'IA industriel.

- Pipeline de données reproductible (24 042 lignes, 15 variables).
- 4 modèles entraines, compares avec rigueur méthodologique.
- Interprétabilité a 3 niveaux (native, permutation, SHAP).
- Dashboard Streamlit opérationnel avec CSS personnalise.
- API FastAPI documentee et validee Pydantic v2.
- Rapport analytique structuré (le présent document).

8.2 Limites identifiees

Le dataset reste un environnement contrôle. En production reelle, des biais supplémentaires apparaîtraient · dérive temporelle des capteurs (drift), hétérogénéité des familles de machines, evenements rares (mode degrade) sous-représentes. Le modèle final gagnerait a etre couple a un système de monitoring statistique de la qualite des données (data drift detection) et a un protocole de re-entrainement périodique.

8.3 Perspectives

- Ajouter une couche temporelle (RNN / LSTM) pour exploiter la succession des mesures.
- Etendre les tâches prédictives a la regression sur la durée de vie restante (rul_hours) et a la classification multi-classe du type de panne.
- Brancher l'API a un système GMAO (Gestion de Maintenance Assistee par Ordinateur) pour déclencher automatiquement les ordres de travail.
- Mesurer l'empreinte carbone reelle des entrainements via CodeCarbon (écoresponsabilité, C4.3).

Annexe A. Alignement RNCP40875 - Bloc BC2

Le tableau ci-dessous detaille la couverture par notre projet de chaque compétence du Bloc 2 (Piloter et implementer des solutions d'IA en s'aidant notamment de l'IA generative).

Code	Compétence	Couverture dans le projet
C3.1	Préparation des données	src/preprocessing.py · ColumnTransformer (SimpleImputer + StandardScaler + OneHotEncoder).
C3.2	Tableau de bord interactif	dashboard/app.py · Streamlit, CSS personnalise, 5 onglets, simulateur temps reel.
C3.3	Analyse exploratoire	scripts/02_eda.py · 7 graphiques + statistiques descriptives.
C4.1	Strategie d'integration IA	Architecture API + Dashboard + Modele decrite section 3.1, schéma 1.
C4.2	Modèles prédictifs ML / DL	src/models.py · 4 modèles dont MLP (DL), pipelines reproductibles et seedés.
C4.3	Évaluation comparative	src/évaluation.py · 6 métriques + ROC/PR + coût computationnel (écoresponsabilité).

References techniques

- scikit-learn 1.x · Pipeline, ColumnTransformer, MLPClassifier.
- XGBoost 2.x · gradient boosting optimise.
- SHAP 0.4x · explicabilite locale et globale.
- Streamlit 1.x · dashboard décisionnel.
- FastAPI 0.1x + Pydantic v2 · API REST validee.
- FPDF2 2.x · production du présent rapport.